

ARCHITECTURE & ENGINEERING REFERENCE

Local Mesh AI

A complete technical reference for the Local Mesh AI system: the Android application and its Kotlin mesh-core module, the Python Desktop Agent and its Mesh API, the local inference backends they orchestrate, and every networking, pairing, streaming and security mechanism that connects them — with per-module file maps and feature-by-feature engineering detail.

EquityAviator · LocalMesh-App

Version 1.0.0-rc.1 — milestones M0–M9 complete

Android (Expo + Kotlin) · Desktop Agent (Python 3.12 + FastAPI) · Control Plane (metadata only)

OCTOBER 2026 · APACHE-2.0

Table of Contents

1. Executive Overview	4
1.1 Delivery status	4
1.2 Design pillars	4
2. System Architecture	5
2.1 Components and responsibilities	5
2.2 Trust boundaries and data classification	6
2.3 Transport tiers	6
2.4 Architecture decision records	7
3. Technology Stack	8
3.1 Desktop Agent (Python 3.12)	8
3.2 Android application	8
3.3 Tooling, protocol and test infrastructure	9
4. Repository and Module Map	9
4.1 Desktop Agent module map (agent/src/localmesh_agent)	9
4.2 Android app module map (apps/mobile/src)	11
4.3 mesh-core Kotlin module map	12
4.4 Protocol, scripts and docs map	12
5. The Desktop Agent (PC Side)	13
5.1 Process model and lifecycle	13
5.2 API layer	13
5.3 Core services	13
5.4 Security subsystem	14
5.5 Adapters	14
5.6 Store schema	14
5.7 Observability and the Doctor	15
6. The Android Application (Device Side)	15
6.1 Structure and navigation	15
6.2 Domain layer: pure, deterministic, tested	15
6.3 Data and state	15
6.4 The mesh-core bridge	16

6.5 mesh-core internals (Kotlin)	16
6.6 packages/mesh-protocol	16
7. Networking and Connectivity	16
7.1 Discovery: mDNS advertisement (§16.2)	16
7.2 Pairing: from QR to device identity (§15.4)	17
7.3 Authentication and tokens	17
7.4 TLS 1.3 with SPKI pinning	17
7.5 Connection management: SM-CONN and the staggered race (§16.1)	17
7.6 Streaming: SSE frames and SM-STREAM (§15.3)	18
7.7 Doctor and the reason ladder (§18.1-§18.4)	19
7.8 Ports and firewall reference	19
8. Security Architecture	19
8.1 Non-negotiables and their enforcement	19
8.2 Identity and key material	20
8.3 Admission control and abuse resistance	20
8.4 Supply-chain and release security	20
9. Feature Engineering Reference (M0-M9)	20
9.1 M0 - Foundations and tooling	20
9.2 M1 - Contracts: OpenAPI-first Mesh API	21
9.3 M2 - Backend adapters and Capability Registry	21
9.4 M3 - Pairing and device identity	21
9.5 M4 - Transport security: TLS 1.3 + SPKI pinning	22
9.6 M5 - Discovery, chat streaming, and connection intelligence	22
9.7 M6 - Control Plane (metadata only)	22
9.8 M7 - Multimodal and durable Tasks	22
9.9 M8 - model:auto routing and agent tools	23
9.10 M9 - Hardening and release engineering	23
9.11 App side - M2-M5 screens and round-13 hardening	23
10. End-to-End System Walkthrough	24
10.1 First run: from zero to first token	24
10.2 The developer loop	24
10.3 What the cloud ever sees	24
11. Testing, QA, and Release Engineering	25

11.1 Test architecture	25
11.2 Quality gates on every change	25
11.3 Release engineering	25
12. Deployment and Operations	26
12.1 Running the Agent	26
12.2 Building the Android app	26
12.3 Operating the mesh	26
12.4 Known open items before 1.0 final	26
13. Open Items and Roadmap	27
Appendix A. API Endpoint Reference	27
A.1 Mesh API (/mesh/v1, TLS 8443)	27
A.2 Admin API (loopback 8444)	28
Appendix B. Glossary	28

1. Executive Overview

Local Mesh AI lets an Android phone drive the large language models that already run on the user's own PC. The Android application (**Local Mesh AI**, package `ai.localmesh.app`) speaks to a Python **Desktop Agent** over the local network or a Tailscale VPN, and the Agent - the only network-facing process on the PC - translates Mesh API calls into requests against local inference engines such as LM Studio, Ollama, or any OpenAI-compatible backend. An optional cloud **Control Plane** mirrors registry metadata and revocation state, but it is architecturally incapable of carrying conversation content. The result is a personal AI mesh in which privacy is a structural property of the design rather than a policy promise.

This document is the complete engineering reference for what has been built. It describes the system architecture, the technology stack of every component, the file-level module map of the monorepo, the internal design of the Desktop Agent and the Android app, the networking and connection machinery (discovery, pairing, TLS pinning, tokens, streaming), the security model, a feature-by-feature engineering reference for milestones M0 through M9, and an end-to-end walkthrough of the whole system in operation. Every statement is grounded in the repository; specification identifiers such as FR-PAIR-03 or §15.3 refer to the governing specification LM-ARCH-001, which ships in the repository at `docs/LocalMesh_AI_Architecture_and_Requirements.md`.

9	10 + 2	17	14/14
App screens (Expo Router)	Kotlin sources + instrumented test files	Mesh API endpoints (OpenAPI)	Release-gate checks passing (M9)

1.1 Delivery status

The project reached **v1.0.0-rc.1** with all ten milestones (M0-M9) complete and the M9 release gate green. The TypeScript side of the mobile monorepo passes 158 tests (141 mobile domain tests plus 17 mesh-protocol tests) under strict TypeScript checking, and the Python agent carries 52 test modules across the four layers defined by §21.5 (unit, contract, integration, security). The Kotlin mesh-core module has been compiled by the Android SDK toolchain into a debug AAR, with two instrumented test classes ready for device or emulator execution. Remaining pre-1.0 items are deliberately owner-gated and are listed in Chapter 13: the on-device penetration-test sign-off, the §18.8 real-network matrix, and approvals for ADR-017/018.

1.2 Design pillars

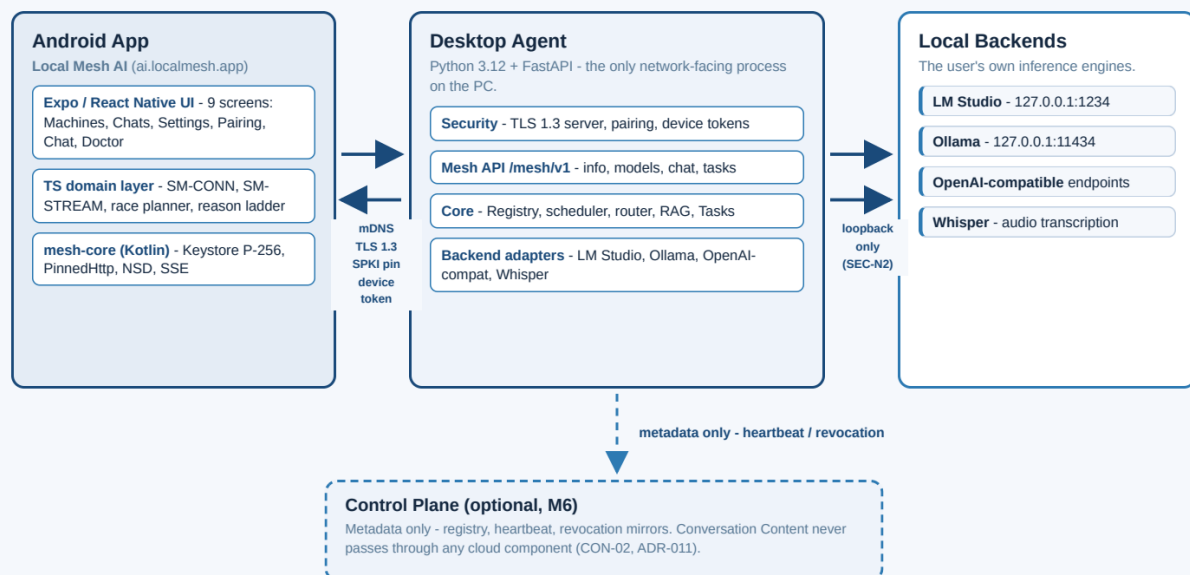
- **Backends stay on loopback.** LM Studio, Ollama, and Whisper are reachable only from the PC itself (SEC-N2). The Agent is the single network-exposed process, which shrinks the attack surface to one hardened service.
- **Content never touches the cloud.** The Control Plane stores metadata only. A database schema that could hold conversation content fails CI automatically (SEC-N3, CON-02, ADR-011, TC-SEC-08).

- **Every call is authenticated.** Network position is never authorization (SEC-N4): each device holds a revocable, opaque Device Token issued during pairing, and the Agent challenges every non-pairing request.
- **Transport is pinned, not merely encrypted.** The app talks TLS 1.3-only with an SPKI pin per Agent; the pin lives in the app database and the private key never leaves the Android Keystore (SEC-N5).
- **Spec-driven development.** Code changes cite specification IDs, contracts are generated from OpenAPI with a CI drift gate (ADR-015), and unknowns become recorded questions rather than guesses.

2. System Architecture

2.1 Components and responsibilities

The system is a three-node mesh with one optional cloud component. The Android app owns the user experience and all connection intelligence; the Desktop Agent owns backends, security, and the Mesh API; the local backends own model inference; and the Control Plane, when enabled, mirrors metadata so that devices can find and re-verify Agents across networks. Figure 1 shows the components and the three trust boundaries between them.



Transport tiers - T0 dev loopback - T1 LAN (mDNS) - T2 Tailscale/VPN - T3 future relay (ADR-005)

Figure 1: System architecture: three trust boundaries - App-Agent (TLS 1.3 + SPKI pin + device token), Agent-backends (loopback only), Agent-Control Plane (metadata only).

Component	Runtime	Responsibility	Source root
Android App	Android (Expo / React Native + Kotlin)	Pairing, connection management, chat UI, tasks and diagnostics; all cryptography keys live in the Android Keystore.	apps/mobile
mesh-core module	Kotlin AAR inside the app	Native primitives: P-256 Keystore identity, TLS 1.3 pinned HTTP, SSE parsing, mDNS discovery (NSD), network monitoring, permissions.	apps/mobile/modules/mesh-core
Desktop Agent	Python 3.12 + FastAPI (uvicorn)	Mesh API /mesh/v1, pairing and device tokens, Capability Registry, scheduler, model:auto routing, RAG, durable Tasks, admin console API, mDNS advertisement.	agent/src/localmesh_agent
Local Backends	LM Studio / Ollama / OpenAI-compatible / Whisper	Model inference. Reached by the Agent over loopback only; never exposed to the network.	external (adapter integration)
Control Plane	SQL schema + migrations (optional)	Agent registry, heartbeat, device revocation mirror. Metadata only by construction (ADR-011, TC-SEC-08).	control-plane
Dashboard	Next.js 16 (dev cockpit)	Local development surface: milestone tracking, admin API panels, virtual-device preview of the app.	src (monorepo root)

Table 1: System components and their source roots.

2.2 Trust boundaries and data classification

The architecture draws exactly three trust boundaries. Boundary one is the radio link between the app and the Agent: every byte crosses TLS 1.3 with an SPKI pin bound to the Agent's identity, and every non-pairing request carries a Device Token. Boundary two is the loopback interface on the PC: backends are bound to 127.0.0.1 and only the Agent may talk to them, so a compromised LAN peer cannot reach a backend directly. Boundary three is the optional uplink to the Control Plane: only metadata (identifiers, heartbeats, revocations) ever crosses it, and the schema itself makes content storage impossible rather than merely forbidden.

Content vs Metadata - the central data-classification rule of the project. **Content** is anything the user says, hears, uploads, or receives: prompts, completions, audio, images, documents. **Metadata** is everything needed to operate the mesh: identifiers, model catalogs, timings, revocations, health. Content stays inside the App-Agent-backend triangle; Metadata may be logged (allow-listed keys only), mirrored to the Control Plane, and observed.

2.3 Transport tiers

Connectivity is modelled as tiers so that the app can rank candidate endpoints and the connection manager can explain exactly why a tier is or is not usable. The tier is part of every stored Endpoint and is reported by the connection state machine.

Tier	Medium	Use in v1
T0	Loopback HTTP (dev only)	Development mode bound to 127.0.0.1 with token auth bypassed (§17.9, QUESTION-105). Off by default (SEC-N6); used by the sandbox run-book and fake-backend scenario.
T1	LAN + mDNS + TLS	The primary path: the Agent advertises <code>_localmesh._tcp</code> on the home network; the app discovers, pins, pairs, and streams over Wi-Fi.
T2	Tailscale / VPN	Remote access for v1 (ADR-005). Tailnet candidates are probed and raced after LAN candidates; the transport interface keeps this tier swappable.
T3	Custom relay (future)	Designed-for but not built: an app-layer end-to-end encrypted relay (Noise XX) so the relay is blind. Requires a new ADR before any work starts.

Table 2: Transport tiers T0-T3. Tiers are stored per endpoint and chosen by the race planner.

2.4 Architecture decision records

Significant choices are recorded as ADRs under `docs/adr/` and referenced from code. The table below lists the decisions most visible in the architecture; the ADR directory is authoritative.

ADR	Decision
ADR-002	Agent service stack: FastAPI + uvicorn + httpx, Pydantic v2 as a contractual boundary.
ADR-005	Remote access via Tailscale (Tier T2) for v1; transport abstraction so T3 relay can slot in later. Cloudflare Tunnel rejected because it terminates TLS at the provider.
ADR-006/007/008	Layered minimal crypto stack: pinned self-signed TLS 1.3, QR one-time-secret pairing, per-device ECDSA P-256 key, opaque tokens. SPAKE2+ reserved for optional manual-code pairing.
ADR-011	Durable Tasks with AES-GCM at rest and 1-hour retention; content-column ban enforced by scan.
ADR-014	Loopback admin listener (port 8444) as a separate FastAPI app for pairing approvals, device management, TLS rotation, and diagnostics.
ADR-015	OpenAPI is the contract; TypeScript types and validators are generated; CI fails on drift.
ADR-017	PEP 517 build backend (setuptools ≥ 77) so the Agent installs as a package.
ADR-018	pynvml replaced by nvidia-ml-py (same module, maintained distribution) after upstream deprecation.
ADR-019	Control Plane activation model: agent heartbeat, register, and revocation-mirror semantics.
ADR-020	Agent runtime tool calling behind a default-deny sandbox (FR-AGENT-RT).
ADR-021	Go/Rust rewrite gate closed: Python stays for 1.0; performance handled by design, not language.

Table 3: Selected architecture decision records.

3. Technology Stack

Versions are pinned in committed lockfiles (`agent/requirements.txt` with hashes, `bun.lock` files) and are never typed from memory - a governance rule that keeps builds reproducible and the audit chain intact. The tables below reflect the pinned state at v1.0.0-rc.1.

3.1 Desktop Agent (Python 3.12)

Layer	Technology	Role
Service	FastAPI 0.142.2 + uvicorn	Mesh API /mesh/v1 on :8443 (TLS 1.3); admin app on :8444 (loopback).
Contracts	Pydantic 2.13.5 (+ pydantic-settings)	Request/response schemas; settings from TOML config (§10.1).
HTTP client	httpx (async)	Backend adapter calls over loopback; SSE parsing from backends.
Crypto	cryptography	TLS identity, ECDSA device keys, AES-GCM task payloads, SPKI pin computation.
Discovery	python-zeroconf	mDNS advertisement of <code>_localmesh._tcp</code> (§16.2). LGPL-2.1-or-later - see THIRD-PARTY-NOTICES.
Hardware probe	psutil (+ nvidia-ml-py optional extra)	CPU/RAM telemetry; NVIDIA GPU probe for capacity hints (§20).
Secrets	keyring	OS keyring storage for Agent-side secrets (SEC-N5).
Quality	ruff, mypy --strict, pytest + pytest-asyncio, import-linter, pip-audit	Lint, strict typing on core/ and security/, four test layers, §10.1 dependency rule, dependency CVE gate.
Packaging	setuptools >= 77 (PEP 517, ADR-017)	Wheel + sdist release artifacts with SHA256SUMS (M9).

Table 4: Desktop Agent technology stack (pinned in `agent/requirements.txt`).

3.2 Android application

Layer	Technology	Role
App shell	Expo SDK ~52, React Native 0.76.9, React 18.3.1 (New Architecture on)	Screens, navigation (expo-router), status bar; web export powers the dashboard's virtual device.
State	zustand 5	Four domain stores: machines, chats, agent data, settings.
Routing	expo-router	Three tabs + pairing, machine detail, chat, and doctor routes.
Language	TypeScript (strict)	Domain layer, data layer, feature modules; tsc --noEmit in the gate chain.
Native module	mesh-core (Kotlin, Gradle 8.10.2, JDK 17)	Keystore P-256, PinnedHttp (TLS 1.3-only + SPKI pin), SseStream, NSD discovery, NetMonitor, permissions.
Protocol	packages/mesh-protocol (generated TS)	Types, validators, and SSE event unions generated from the OpenAPI spec (ADR-015).
Persistence	App store SQLite (schema verbatim §14.2)	Encrypted at rest per §11.5; key wrapping lives in the native layer; content-column discipline mirrors SEC-N3.

Table 5: Android application stack.

3.3 Tooling, protocol and test infrastructure

- **Monorepo tooling:** bun for JavaScript workspaces and test running (`bun test apps/mobile packages/mesh-protocol`); Python venv with pinned hashed requirements for the agent.
- **Contract chain:** `docs/openapi/mesh-v1.json` is regenerated by `scripts/export_openapi.py`; `packages/mesh-protocol` is generated from it; `scripts/check_openapi_drift.py` fails CI on any divergence (ADR-015, NFR-MAINT-01).
- **Security scanners:** cleartext-manifest scan (SEC-N1), content-column scan (SEC-N3), secret scan (SEC-N5), pip-audit and npm audit - all wired into the PR gate pipeline.
- **Test backends:** `tools/fake-backends` ships dependency-free fakes for LM Studio (:1234), Ollama (:11434), and Whisper, used by contract and integration layers and by the sandbox run-book.
- **Dashboard:** Next.js 16 + Tailwind + shadcn/ui used as the development cockpit, including a virtual-device panel that renders the app's web export against the real Agent.

4. Repository and Module Map

The repository is a single monorepo so that contracts, agent, app, and governance docs move in lockstep. This chapter is the file-level map: every module, what it contains, and why it exists. Paths are relative to the repository root.

Path	Contents
<code>agent/</code>	Desktop Agent package (src + tests + pyproject + pinned requirements).
<code>apps/mobile/</code>	Expo app: <code>app/</code> routes, <code>src/</code> domain+data+state+features+infra+ui, <code>android/</code> committed prebuild, <code>modules/mesh-core</code> Kotlin module.
<code>packages/mesh-protocol/</code>	Generated TS types/validators/SSE events + generation script + drift tests.
<code>control-plane/</code>	Optional metadata-only schema (<code>migrations/0001_init.sql</code>) + content scan script.
<code>tools/fake-backends/</code>	Stdlib-only fake LM Studio / Ollama / Whisper + their tests.
<code>scripts/</code>	pytest layers, OpenAPI export/drift, security scanners, release gate/build, pairing smoke.
<code>docs/</code>	LM-ARCH-001 spec, ADRs, open questions, release, run-book, handover, OpenAPI, CI staging, security checklists.
<code>src/</code> (root)	Next.js dashboard (development cockpit).
<code>AGENTS.md</code> / <code>worklog.md</code>	Governance digest and the running engineering ledger.

Table 6: Monorepo top-level layout.

4.1 Desktop Agent module map (agent/src/localmesh_agent)

File / package	Purpose
__main__.py, cli.py	Entry point: <code>python -m localmesh_agent run [--config ...] [--dev-insecure-loopback]</code> ; CLI plumbing.
app.py	FastAPI application factory and lifespan: store migrations, registry refresh, mDNS start, Control Plane registration, keep-warm, admin app mount.
config.py	Pydantic-settings TOML config: listen (8443) / admin (8444) ports, mDNS settings, backend endpoints, retention, dev-mode flags.
admin_app.py	Separate loopback admin API: pairing open/approve/deny/close, devices, TLS rotate + backup pin, status, doctor, metrics, Control Plane register/status.
doctor.py	§18.4 ordered connectivity checks; feeds the app's Doctor screen and reason ladder.
api/deps.py, api/errors.py	Auth dependencies (device-token challenge) and the RFC-style error model.
api/v1/*.py	One router per resource: info, health, device, pair, auth, models, chat, requests, tasks (§13 contracts).
core/registry.py	Capability Registry: backend/model catalog with §16.3 merge rules and model_cache persistence.
core/scheduler.py	§16.4 admission control for generation requests; queue semantics.
core/agent_loop.py	Chat orchestration: prompt assembly, backend call, SSE frame emission, cancellation.
core/router.py	§16.6 model:auto scoring engine (explainability in mesh.meta); FR-RTE flag off by default.
core/tasks.py	Durable Tasks: execution, SSE fan-out, §13.9 retention sweep (1 h), encrypted attachments.
core/rag.py	Local retrieval: ingestion into rag_sources/sections/vectors, query path (FR-MM-03).
core/tools.py	ADR-020 default-deny tool sandbox for agent runtime tools.
core/warm.py	§16.5 keep-warm chain (P1): model stay-loaded policies.
core/policy.py	Policy checks incl. M7 multimodal limits (test_policy_m7).
core/control_plane.py	Heartbeat/register loop and revocation mirror against the Control Plane (ADR-019).
core/entities.py, core/errors.py	Domain entities and typed errors shared across services.
security/tls.py	TLS 1.3 serving identity: self-signed cert, SPKI computation, reload/rotate support.
security/pairing.py	SM-PAIR (§15.2): pairing sessions, one-time secret verification, FR-PAIR-03 zero-body completion.
security/manual_pairing.py	Optional SPAKE2+ manual-code pairing path (P2) with test vectors.
security/tokens.py, devices.py	Opaque device-token issuance/hashing/refresh semantics; device lifecycle and revocation.
security/crypto.py, task_crypto.py	ECDSA/AES-GCM primitives; AES-GCM at-rest encryption for task payloads and RAG sections.
security/ratelimit.py	Per-device rate limiting and lockout (TC-SEC-09).
adapters/backends/*.py	lmstudio, ollama, openai_compat, whisper: model listing, load/unload, streaming chat, transcription.
adapters/discovery/mdns.py	§16.2 advertisement with TXT hints (aid, n, fp, api, po) and 60 s watchdog.
adapters/controlplane.py	Control Plane client for register/heartbeat/revocation mirror.
adapters/hardware/nvidia_probe.py, psutil_probe.py	Capacity telemetry (§20) with graceful degradation.

File / package	Purpose
adapters/tailscale.py	T2 probe: Tailnet status and candidate addresses (WP-14).
adapters/keyring_store.py, ports.py	OS keyring integration; port interfaces that keep §10.1 dependency direction.
store/sqlite.py, store/migrations/*	SQLite store with migrations 0001_init.sql and 0002_tasks.sql.
observability/logging.py, redact.py, metrics.py	Allow-list logging (§17.10), redaction, and §20.1 metrics registry.

Table 7: Agent module map - every source file and its responsibility.

4.2 Android app module map (apps/mobile/src)

File / package	Purpose
domain/entities.ts	Core types: Agent, Endpoint, Tier, Device, Message, PermState.
domain/connection/sm-conn.ts	§15.1 connection state machine - pure reducer over 11 states.
domain/connection/race-planner.ts	§16.1 staggered race across candidate endpoints with cancellation.
domain/connection/reasons.ts	§18.3 reason ladder: probe failure aggregation to ReasonCode + CTA.
domain/connection/clock.ts	Simulated clock interface so timers are deterministic in tests.
domain/sm-stream.ts	§15.3 per-generation stream reducer: chunks, meta, stats, errors, cancel.
domain/tokens.ts	Token lifecycle: store, refresh at 80% of expires_in, one refresh on 401.
data/mesh/client.ts, data/mesh/sse.ts	HTTP + SSE client over the mesh-core bridge; frame normalization.
data/db/schema.ts, data/db/repositories.ts	§14.2 schema verbatim + repositories; encrypted SQLite, no content columns beyond messages.content.
state/*.ts	zustand stores: machines, chats, agentData, settings + root store wiring.
features/pairing/qr.ts, confirm.ts	QR scan payload parsing and pairing confirmation flow.
features/machines/live.ts, view.ts	Machine list/selection logic and live status derivation.
features/chat/send.ts, pipeline.ts, composer.ts	Send pipeline, streaming render pipeline (§16.7), composer state.
features/models/view.ts	Model catalog view-model from the Capability Registry.
features/diagnostics/doctor.ts, live.ts	Doctor screen logic walking §18.4 checks; live probe state.
features/settings/export.ts	Settings and data export.
infra/meshCore.ts, infra/webMeshCore.ts	Bridge to the Kotlin module; web implementation so the dashboard can preview the app.
ui/theme.ts, ui/useThemeMode.ts, ui/components.tsx	Design tokens, dark/light mode, shared components.
app/(tabs)/*.tsx, app/pair/*.tsx, app/machine/[agentId].tsx, app/chat/[conversationId].tsx, app/doctor.tsx	The nine screens: Machines, Chats, Settings tabs; pair scan/confirm; machine detail; chat; doctor.

Table 8: Android app module map (TypeScript side).

4.3 mesh-core Kotlin module map

File	Purpose
Keystore.kt	P-256 device key pair inside the Android Keystore; alias per Agent; sign/verify support for pairing and challenges.
PinnedHttp.kt	TLS 1.3-only HTTP client: Tls13OnlySocketFactory, PinningTrustManager, AllowAllButPinnedVerifier, PinnedTarget; enforces the stored SPKI pin and FR-PAIR-03 zero-body pairing completion.
SseStream.kt	Incremental SSE frame parser with timeout and reconnect semantics for /chat/completions and task events.
Nsd.kt	NSD-based discovery of _localmesh._tcp with resolve callbacks (mirrors §16.2 on Android).
NetMonitor.kt	Network callback wrapper: wifi/VPN changes feed SM-CONN networkChanged events.
PairPayload.kt	QR payload codec for the one-time Pairing Secret + Agent hints.
Permissions.kt	LocalNetworkPermissions: runtime permission orchestration for NEARBY_WIFI_DEVICES et al.
MeshCoreModule.kt	Expo module definition exposing the above to TypeScript (expo-modules-core).
MeshCoreException.kt, B64Url.kt	Typed errors and base64url helpers shared by the bridge.
androidTest: PinnedHttpInstrumentedTest.kt, NsdInstrumentedTest.kt	On-device tests: pin-mismatch rejection (FR-PAIR-03), TLS 1.3-only handshake, discovery resolve.

Table 9: mesh-core Kotlin sources; compiled artifact mesh-core-debug.aar verified in CI sandbox.

4.4 Protocol, scripts and docs map

Path	Purpose
packages/mesh-protocol/src/types.ts, validators.ts	Generated request/response types and runtime validators (ADR-015).
packages/mesh-protocol/src/sse-events.ts	Discriminated union of SSE frames: default chunk, mesh.meta, mesh.stats, mesh.error.
packages/mesh-protocol/scripts/generate.ts	Regenerates the package from docs/openapi/mesh-v1.json.
scripts/check_openapi_drift.py	Fails if the committed OpenAPI diverges from the live app (ADR-015).
scripts/pytest_layer.sh	Runs one §21.5 test layer with the right flags.
scripts/security/scan_cleartext_manifest.py	SEC-N1: fails on cleartext outside debug source sets.
scripts/security/scan_content_columns.py	SEC-N3/CON-02: fails on schema columns able to hold Content outside sanctioned tables.
scripts/security/scan_secrets.py	SEC-N5 tripwire for committed keys/tokens.
scripts/release_gate.sh, release_build.sh	M9: 14-check release gate; wheel/sdist + SHA256SUMS + artifact scan (TC-SEC-07).
docs/ci/PR-GATE.yml (+ README)	Staged GitHub Actions pipeline (full §21.4 chain) with restore instructions.

Path	Purpose
docs/handover/RUN-BOOK.md, AGENT-BRIEF.md	Verified run recipes and agent context bundle.
docs/OPEN_QUESTIONS.md, docs/RELEASE.md	Owner decisions ledger; release engineering + M9 decision register.

Table 10: Protocol, tooling and documentation map.

5. The Desktop Agent (PC Side)

5.1 Process model and lifecycle

The Agent is a single Python process hosting two FastAPI applications. The Mesh API binds TLS on port **8443** on all interfaces (or the Tailnet address) and serves `/mesh/v1`; the admin app binds **8444** strictly on 127.0.0.1 and exposes owner operations that never need to leave the machine. Startup follows a deterministic lifespan: open the store and run migrations, load configuration, compute the TLS identity and SPKI pin, start the Capability Registry refresh, advertise mDNS, register with the Control Plane when enabled, and start the keep-warm and retention loops. Shutdown reverses the chain and flushes audit events.

```
python -m localmesh_agent run [--config config.toml] [--dev-insecure-loopback]
```

SEC-N6 in practice: `--dev-insecure-loopback` is the only dev path (plain HTTP on 127.0.0.1, auth bypassed, QUESTION-105 recorded). The default is TLS 1.3; the release gate fails if dev paths are reachable in a release build.

5.2 API layer

Routers live under `api/v1/`, one file per resource, and mirror §13 of the specification byte-for-byte through the OpenAPI drift gate. `api/deps.py` resolves the device token on every non-pairing route; `api/errors.py` renders the canonical error envelope so the app's reason ladder can map failures deterministically. The full endpoint list with methods is in Appendix A; chat is the only streaming route, emitting SSE frames on the same POST response.

5.3 Core services

The core package implements the Agent's brain. The **Capability Registry** merges live backend catalogs with cached rows (§16.3) so the app sees a stable `mesh_model_id` space of the form `::`. The **scheduler** (§16.4) admits generation requests and prevents model thrash; the **keep-warm** chain (§16.5) implements stay-loaded policies for P1 latency. The **agent loop** orchestrates one chat generation: it resolves the model, calls the adapter, translates backend deltas into Mesh SSE frames, honors cancellation, and closes with usage statistics. The **router** (§16.6) implements `model:auto` as a explainable scoring engine over registry metadata - gated behind a flag that ships off by default (FR-RTE-01..03). **Tasks** are durable workloads with attachments

encrypted at rest (AES-GCM, ADR-011), a retention sweeper that keeps results at most one hour (§13.9), and per-task SSE event streams. **RAG** (FR-MM-03) ingests documents into encrypted sections and vector rows for local retrieval. **Tools** (ADR-020) run behind a default-deny sandbox so runtime tool calling can be enabled safely. The **Control Plane** client performs heartbeat, registration, and revocation mirroring per ADR-019.

5.4 Security subsystem

The security package owns identity and admission. `tls.py` maintains the serving certificate and SPKI pin, including rotation with a backup pin window via the admin API. `pairing.py` runs SM-PAIR (§15.2): a pairing session opens via the admin console, the Agent renders a QR containing a one-time Pairing Secret, and completion is verified against that secret with the zero-body trust binding of FR-PAIR-03. `manual_pairing.py` offers the optional SPAKE2+ code path (P2) with published test vectors. `tokens.py` issues opaque Device Tokens - only hashes are stored - and `devices.py` manages lifecycle and revocation, which tears down in-flight streams (§15.6). `ratelimit.py` enforces per-device limits with lockout (TC-SEC-09), and `task_crypto.py` seals task payloads with AES-GCM.

5.5 Adapters

Adapters isolate the outside world behind the port interfaces of `adapters/ports.py`, keeping the §10.1 dependency rule (core never imports adapters directly; composition happens at the app layer). Backend adapters translate the OpenAI-compatible surface of LM Studio (127.0.0.1:1234), Ollama (127.0.0.1:11434), generic OpenAI-compatible servers, and Whisper into internal entities, including SSE delta parsing that is unit-tested against recorded fixtures. The mDNS adapter implements §16.2 exactly; the hardware probes expose capacity telemetry with graceful degradation when GPUs are absent; the Tailscale adapter probes Tailnet state to produce Tier T2 candidates.

5.6 Store schema

Table (migration)	Holds
agent_identity (0001)	Agent UUID, display name, TLS identity material references.
devices (0001)	Paired devices: id, public key, display name, state.
tokens (0001)	Hashed opaque device tokens, expiry, refresh family.
audit_events (0001)	Append-only metadata audit log - never Content.
backends (0001)	Configured backends and their loopback endpoints.
model_cache (0001)	Cached model catalog rows feeding the Registry merge.
settings (0001)	Key/value operational settings.
tasks, task_files (0002)	Durable tasks and their AES-GCM sealed attachments.
rag_sources, rag_sections, rag_vectors (0002)	Local RAG corpus: sources, encrypted sections, embeddings.

Table 11: Agent SQLite schema (store/migrations).

5.7 Observability and the Doctor

Logging follows the §17.10 allow-list: keys that may appear in logs are enumerated, everything else is dropped, and `observability/redact.py` scrubs accidental content-shaped fields - a unit test (`test_logging_allowlist`) pins this. Metrics (§20.1) count requests, stream outcomes, scheduler admissions, and backend latencies, surfaced both on the admin API and the dashboard. The Doctor (§18.4, `doctor.py`) runs the ordered connectivity checks - permissions, discovery, reachability per tier, TLS pin, auth, backends - and returns machine-readable results that the app aggregates through the §18.3 reason ladder into a single actionable diagnosis.

6. The Android Application (Device Side)

6.1 Structure and navigation

The app is an Expo SDK 52 / React Native 0.76.9 application with the New Architecture enabled, navigating with `expo-router`. Three tabs - Machines, Chats, Settings - cover daily use; stack routes handle pairing (scan, confirm), machine detail, the chat conversation, and the Doctor. A dark/light theme system (`ui/theme.ts`, `useThemeMode.ts`) follows the OS setting. The same codebase exports to web (`react-native-web`), which is how the dashboard's virtual-device panel renders the real UI against a real Agent during development.

6.2 Domain layer: pure, deterministic, tested

All connection and streaming intelligence lives in a pure TypeScript domain layer with no imports from React, the bridge, or the network. Timers come from an injected simulated clock (`domain/connection/clock.ts`), so tests assert exact timings. **SM-CONN** (§15.1) is a reducer over eleven states: `IDLE`, `PLANNING`, `CONNECTING`, `AUTHENTICATING`, `CONNECTED`, `DEGRADED`, `RECONNECTING`, `BLOCKED_PERMISSION`, `UNREACHABLE`, `PIN_MISMATCH`, and `REVOKED`; it consumes inputs such as app foreground/background, networkChanged, discovery events, permission state, probe results, and stream errors, and outputs the selected Endpoint and tier. **SM-STREAM** (§15.3) owns one chat generation from POST to final token. The **race planner** (§16.1) staggers candidate endpoints and cancels losers. The **reason ladder** (§18.3, `reasons.ts`) folds probe failures into one ReasonCode with a call to `action`. **Token management** (`domain/tokens.ts`) refreshes at 80% of lifetime (`REFRESH_AT_FRACTION = 0.8`) or once on a 401, and the §11.3 purity tests keep coverage of this layer at or above the spec floor.

6.3 Data and state

The data layer splits transport (`data/mesh/client.ts`, `sse.ts`) from persistence (`data/db/`). The app store carries the §14.2 schema verbatim - agents, conversations, messages, and friends - with the SPKI pin and ordered endpoints stored per agent; encryption at rest is §11.5 with key wrapping in the native layer, and the

content-column discipline mirrors SEC-N3 on the device. Presentation state lives in four zustand stores (machines, chats, agentData, settings) kept deliberately thin: they select from domain state rather than duplicating it.

6.4 The mesh-core bridge

TypeScript never touches sockets or keys directly; it calls the Kotlin module through `infra/meshCore.ts`. The Kotlin side (below) exposes key generation and signing, pinned HTTP with streaming, NSD discovery, and network monitoring. A parallel `infra/webMeshCore.ts` implements the same surface for the browser so the dashboard's virtual device exercises the identical domain logic against a real Agent - the bridge is the only platform-divergent seam in the app.

6.5 mesh-core internals (Kotlin)

`PinnedHttp.kt` is the security-critical class: a TLS 1.3-only socket factory rejects every protocol below 1.3, and a custom trust manager verifies the leaf SPKI against the stored pin - certificate authorities are irrelevant on this path. The pairing request enforces FR-PAIR-03's zero-body trust binding. `Keystore.kt` generates the P-256 device key non-exportably inside Android Keystore under a per-Agent alias; `SseStream.kt` parses frames incrementally with timeout and retry semantics; `Nsd.kt` mirrors §16.2 discovery; `NetMonitor.kt` converts platform network callbacks into SM-CONN events; `PairPayload.kt` codes the QR; and `Permissions.kt` orchestrates local-network runtime permissions. Two instrumented test classes (pin mismatch, NSD resolve) run on devices or emulators; the module compiles to `mesh-core-debug.aar`, verified with the Android SDK toolchain during round 13.

6.6 packages/mesh-protocol

The protocol package is generated, never hand-edited: `scripts/generate.ts` reads `docs/openapi/mesh-v1.json` and emits `types.ts`, `runtime validators.ts`, and the SSE event union in `sse-events.ts` (default chunk, mesh.meta, mesh.stats, mesh.error). A generated-snapshot test and the repo-level drift gate make contract drift a visible, one-line diff - the app, the dashboard, and the tests all consume this single source of truth.

7. Networking and Connectivity

Connectivity is the hardest part of a local-first mesh, and it is engineered as a ladder: every layer must hold - permissions, discovery, reachability, TLS pin, authentication, backends - and when something breaks, the same ladder explains it top-down. This chapter walks the ladder.

7.1 Discovery: mDNS advertisement (§16.2)

The Agent advertises service type `_localmesh._tcp.local.` with instance name `()` on the listen port. TXT records carry bounded hints (each at most 255 bytes): `v=1`, `aid=`, `n=`, `fp=`, `api=v1`, and `po=0|1` mirroring

pairing_open. TXT is an untrusted hint (T-21): the authoritative pairing-open check is GET /mesh/v1/info, refreshed by a 60-second watchdog, and the full SPKI pin is verified by the TLS handshake, not by TXT.

7.2 Pairing: from QR to device identity (§15.4)

- **1. Owner opens pairing** on the PC (admin console or CLI): a pairing session (SM-PAIR) starts and renders a QR containing the one-time Pairing Secret plus Agent hints.
- **2. App scans** (pair/scan) and shows a confirmation card (pair/confirm) with the Agent fingerprint; the user confirms.
- **3. Trust binding:** the app connects over TLS 1.3, verifies the Agent pin, generates its P-256 device key in the Keystore, and completes pairing with zero request body - the QR secret plus the pinned channel carry the proof (FR-PAIR-03).
- **4. Owner approves** on the PC; the Agent persists the device and issues an opaque Device Token (only the hash is stored).
- **5. App stores** agent record: SPKI pin, device id, key alias, ordered endpoints - and is thereafter a first-class mesh member.

7.3 Authentication and tokens

Every non-pairing call presents the Device Token. Token bundles carry an expiry; the app schedules refresh at 80% of expires_in and falls back to a single refresh on a 401, so streaming calls never dead-loop on auth. Revocation is immediate: the Agent mirrors revocations (and, when enabled, mirrors them to the Control Plane), and an in-flight stream is torn down per §15.6 with the stream reducer emitting a terminal REVOKED reason rather than a generic error.

7.4 TLS 1.3 with SPKI pinning

The Agent serves TLS 1.3 with a self-signed certificate whose identity is not a certificate authority chain but a single pinned key: the app stores `b64url(SHA-256(SPKI))` per agent at pairing time and the Kotlin `PinningTrustManager` accepts exactly that leaf key. The TLS 1.3-only socket factory makes downgrade negotiation impossible. Rotation is a supported operation: the admin API rotates the serving certificate while continuing to accept the backup pin for a grace window, so paired devices re-pin without re-pairing. Instrumented tests assert both the handshake floor and the pin-mismatch rejection path.

7.5 Connection management: SM-CONN and the staggered race (§16.1)

Given the ordered endpoint list (LAN first, Tailnet next), the race planner starts probes with increasing stagger delays and cancels losers as soon as one endpoint authenticates cleanly. The reducer transitions through PLANNING and CONNECTING/AUTHENTICATING into CONNECTED; failures land in DEGRADED (with a 10-second re-plan timer), UNREACHABLE, PIN_MISMATCH, BLOCKED_PERMISSION, or REVOKED - each a distinct user-facing diagnosis rather than a generic timeout. NetMonitor turns Wi-Fi and VPN changes into networkChanged events so the manager re-plans

before the user notices.

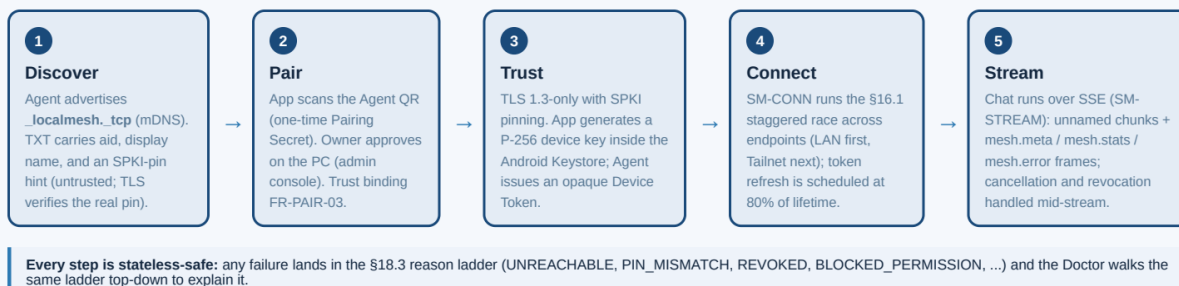


Figure 2: Connection lifecycle: discover, pair, trust, connect, stream - with the reason ladder explaining every failure mode.

7.6 Streaming: SSE frames and SM-STREAM (§15.3)

Chat is a single POST to `/mesh/v1/chat/completions` whose response is an SSE stream. The Agent normalizes backend deltas into Mesh frames: unnamed events carry OpenAI-style `chat.completion.chunk` deltas; `mesh.meta` explains routing and model resolution; `mesh.stats` closes the stream with usage; `mesh.error` carries typed failures. On the device, `SseStream` parses incrementally, `SM-STREAM` reduces frames into message state, and the §16.7 render pipeline throttles UI updates so long generations stay smooth. Cancellation uses `DELETE /mesh/v1/requests/{request_id}` or fires automatically on app background.

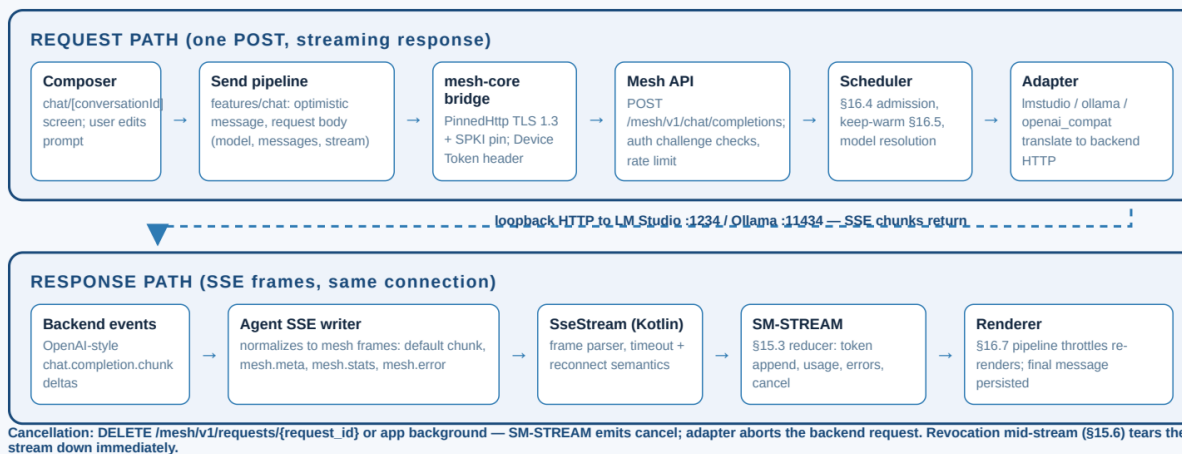


Figure 3: Chat streaming data path: request lane (composer to adapter) and response lane (backend deltas back to the renderer), with cancellation and revocation semantics.

7.7 Doctor and the reason ladder (§18.1-§18.4)

When anything fails, the Doctor screen walks the §18.4 ordered checks and aggregates failures through §18.3 into one ReasonCode - for example a pin mismatch after an Agent certificate rotation without a grace window, or BLOCKED_PERMISSION after an OS update revokes local-network access. Each reason maps to a concrete call to action in the UI, which is what makes the mesh debuggable by non-experts. The §18.8 network matrix (LAN, Tailnet, firewall-on, VPN-conflict) is the pre-release script for validating this behavior on real hardware.

7.8 Ports and firewall reference

Port	Bound to	Purpose
8443	All interfaces (LAN/Tailnet)	Mesh API /mesh/v1 over TLS 1.3 - the only port that must be reachable from devices.
8444	127.0.0.1 only	Admin API: pairing approvals, device management, TLS rotation, doctor, metrics.
1234	127.0.0.1 only	LM Studio default; loopback-only by SEC-N2.
11434	127.0.0.1 only	Ollama default; loopback-only by SEC-N2.
Tailscale	managed by Tailscale	Tier T2 tunneling; the mesh never opens extra ports for it (ADR-005).

Table 12: Port and firewall reference (§18.7).

8. Security Architecture

8.1 Non-negotiables and their enforcement

Rule	Meaning	Enforced by
SEC-N1	No cleartext HTTP on non-loopback; no cleartext in release builds.	scan_cleartext_manifest.py: cleartext allowed only in src/debug source sets (§17.9); release scanner hard-fails.
SEC-N2	Backends reachable via loopback only; Agent is the only exposed process.	Adapter configuration + deployment docs; firewall reference §18.7.
SEC-N3	No Content in logs, crash reports, analytics, or Control Plane.	Allow-list logging + redact.py; scan_content_columns.py; TC-SEC-08 CP schema scan.
SEC-N4	LAN/Tailnet position is never authorization.	Device-token challenge on every non-pairing route (api/deps.py).
SEC-N5	Secrets stay in their store; never logged.	Keystore/keyring bindings; secret scan; single exception is the one-time QR Pairing Secret.
SEC-N6	No dev/insecure mode on by default.	Flag default-off; release gate fails if dev paths are reachable (--release).

Table 13: Security non-negotiables (AGENTS.md) and enforcement points.

8.2 Identity and key material

Three key classes exist. The **Agent TLS identity** is a self-signed certificate whose SPKI hash is the pin every device stores; rotation is supported with a backup-pin grace window. The **device identity** is a P-256 ECDSA key generated inside the Android Keystore and never exported; pairing binds it to the Agent's device record. The **Device Token** is an opaque bearer secret, stored hashed server-side and refreshed at 80% lifetime client-side. Task payloads and RAG sections are sealed with AES-GCM under Agent-held keys (ADR-011), and Agent-side secrets live in the OS keyring.

8.3 Admission control and abuse resistance

Pairing requires a physically-present owner on both ends: the QR secret lives only until the session closes, and the admin console approves or denies each candidate. After pairing, per-device rate limits with lockout (TC-SEC-09) blunt token brute force, replay protection (TC-SEC-03) covers challenge exchange, and the fuzz layer (TC-SEC-05) hammers the parsers. The SPAKE2+ manual path uses published test vectors (test_spake2_vectors) so the protocol implementation is pinned to the standard.

8.4 Supply-chain and release security

- **Pinned, hashed dependencies:** agent/requirements.txt carries hashes (§17.11); bun.lock pins the JS side; no dependency enters without an ADR.
- **Audit gates:** pip-audit for Python, npm audit for the mobile project (omit dev), both wired into the PR gate pipeline.
- **Secret hygiene:** scripts/security/scan_secrets.py tripwire scans tracked files on every PR (SEC-N5).
- **Artifact integrity:** release_build.sh produces wheel + sdist + SHA256SUMS and runs the TC-SEC-07 artifact scan; signing is manual (minisign) with keys outside the repository by design.
- **Release gate:** scripts/release_gate.sh runs 14 checks in §21.4 order and enforces the pentest sign-off requirement for --release.

9. Feature Engineering Reference (M0-M9)

Each milestone below lists its goal, the primary files that deliver it, and how it works. Together they form the feature map of v1.0.0-rc.1; the milestone definitions come from §22.1 and the delivery ledger lives in worklog.md.

9.1 M0 - Foundations and tooling

Goal. Stand up the monorepo, governance, CI gate scaffolding, and dependency-free test backends.

Key files. pyproject/requirements tooling; scripts/pytest_layer.sh; scripts/security/*; tools/fake-backends/*; docs/ governance set.

How it works. The fake backends speak enough of the LM Studio, Ollama, and Whisper HTTP surfaces for all later layers to test against recorded reality instead of mocks; the CI pipeline stages the \$21.4 gate order with announced activations so each later milestone flips gates live.

Verified by. Fake-backend tests; gate scaffolding executed in CI from M0 onward.

9.2 M1 - Contracts: OpenAPI-first Mesh API

Goal. Define the entire Mesh API as a versioned OpenAPI document and generate client contracts from it.

Key files. docs/openapi/mesh-v1.json; scripts/export_openapi.py; scripts/check_openapi_drift.py; packages/mesh-protocol/*.

How it works. The FastAPI app is the source of the spec; the script regenerates the committed JSON, the TS package is generated from that JSON, and the drift check proves both directions on every PR - contract drift becomes a visible one-line diff instead of a runtime surprise (ADR-015).

Verified by. generated.test.ts (17 mesh-protocol tests); drift gate green in CI.

9.3 M2 - Backend adapters and Capability Registry

Goal. Integrate LM Studio, Ollama, OpenAI-compatible servers, and Whisper behind one stable model namespace.

Key files. adapters/backends/{lmstudio,ollama,openai_compat,whisper}.py; core/registry.py; store model_cache.

How it works. Adapters normalize list/load/unload/chat/transcribe across backends; the Registry merges live catalogs with cached rows (§16.3) so mesh_model_id values stay stable across restarts and backend hiccups.

Verified by. contract tests vs recorded fixtures; integration tests against fake backends; unit tests for merge rules.

9.4 M3 - Pairing and device identity

Goal. Turn a QR scan into a persistent, revocable device trust relationship.

Key files. security/pairing.py, manual_pairing.py, tokens.py, devices.py; api/v1/pair.py, auth.py; admin_app.py pairing routes.

How it works. SM-PAIR (§15.2) manages session state (open, scanned, approved, closed); the QR carries the one-time secret; completion is zero-body (FR-PAIR-03); approval issues an opaque token whose hash alone is stored. SPAKE2+ manual pairing is available as the fallback path.

Verified by. test_pairing, test_manual_pairing (with SPAKE2+ vectors), test_tokens, integration test_pairing_auth.

9.5 M4 - Transport security: TLS 1.3 + SPKI pinning

Goal. Make the wire tamper-proof and MITM-proof without any certificate authority.

Key files. security/tls.py; mesh-core PinnedHttp.kt (+ instrumented tests); adapters/tailscale.py (T2 probes).

How it works. The Agent serves TLS 1.3-only; devices pin the SPKI at pairing and reject mismatches (PIN_MISMATCH reason); rotation uses a backup-pin grace window; Tailnet candidates are probed and ranked for Tier T2.

Verified by. test_tls, test_tc_sec_10_tls13, PinnedHttpInstrumentedTest (pin mismatch), test_tailscale.

9.6 M5 - Discovery, chat streaming, and connection intelligence

Goal. Deliver the day-one user flow: find the Agent, connect, and stream a conversation.

Key files. adapters/discovery/mdns.py; api/v1/chat.py, requests.py; core/agent_loop.py; app domain: sm-conn, sm-stream, race-planner, clock; data/mesh/*; mesh-core SseStream.kt, Nsd.kt, NetMonitor.kt.

How it works. mDNS (§16.2) advertises with bounded TXT hints; the app races endpoints with stagger (§16.1); chat POSTs stream SSE frames that SM-STREAM reduces; cancellation is a DELETE on the request id; NetMonitor re-plans on network change; every failure lands on the reason ladder.

Verified by. test_mdns, test_sse_parser, test_scheduler, test_stream_timeouts; integration test_chat_sse, test_stream_robustness; 141 mobile domain tests incl. state machines.

9.7 M6 - Control Plane (metadata only)

Goal. Add optional cloud coordination without creating any content path.

Key files. control-plane/migrations/0001_init.sql; core/control_plane.py; adapters/controlplane.py; admin routes for CP register/status.

How it works. The Agent heartbeats, registers its public metadata, and mirrors revocations (ADR-019); the schema scan TC-SEC-08 proves no column can hold Content, making CON-02 structural rather than procedural.

Verified by. test_control_plane, integration test_control_plane_api, security test_tc_sec_08_cp_schema.

9.8 M7 - Multimodal and durable Tasks

Goal. Support vision/audio inputs, transcription, local retrieval, and long-running work.

Key files. core/tasks.py, rag.py, policy.py (M7 rules); security/task_crypto.py; store/migrations/0002_tasks.sql; adapters/backends/whisper.py; api/v1/tasks.py.

How it works. Multimodal parts ride the chat contract (§13.6); Whisper transcribes audio (FR-MM-02); documents are embedded into encrypted RAG sections/vectors (FR-MM-03); tasks persist with

AES-GCM-sealed payloads and a one-hour retention sweep (§13.9) with per-task SSE events.

Verified by. test_policy_m7, test_rag, test_task_crypto, integration test_tasks_api; policy tests pin the limits.

9.9 M8 - model:auto routing and agent tools

Goal. Let the mesh choose models explainably and run tools safely.

Key files. core/router.py, tools.py; api chat meta frames (mesh.meta routing).

How it works. The §16.6 rule engine scores registry metadata to pick a model for model:auto and explains the choice in mesh.meta; the feature ships behind a flag, off by default (FR-RTE-01..03). Tool calling executes only inside the ADR-020 default-deny sandbox (FR-AGENT-RT).

Verified by. test_router, test_router_auto, integration test_auto_routing, test_tools_runtime.

9.10 M9 - Hardening and release engineering

Goal. Make the project releasable: gates, artifacts, and the rc.1 tag.

Key files. scripts/release_gate.sh, release_build.sh; docs/RELEASE.md; docs/CHANGELOG.md; packaging/.

How it works. The gate runs 14 checks in §21.4 order (tests, scanners, SEC-N6, pentest sign-off requirement); release_build produces wheel + sdist + SHA256SUMS and scans artifacts (TC-SEC-07); the outcome was v1.0.0-rc.1 with the Go/Rust gate closed (ADR-021).

Verified by. test_release_engineering, test_tc_sec_07_release_manifest; release gate 14/14 PASS.

9.11 App side - M2-M5 screens and round-13 hardening

Goal. Ship the Android surface for the mesh: pairing, machines, chat, doctor.

Key files. app/* screens; features/*; domain/*; data/*; infra/*; mesh-core Kotlin (compiled to AAR).

How it works. Round 12 landed the M2-M5 app surface against the green domain tests; round 13 compiled the Kotlin module with the Android SDK (six Kotlin fixes), wired the live virtual-device preview into the dashboard, applied the app name Local Mesh AI (ai.localmesh.app), fixed SM-STREAM state completion, and swept dead code.

Verified by. bun test apps/mobile packages/mesh-protocol = 158 passing; strict tsc; mesh-core AAR build verified.

10. End-to-End System Walkthrough

This chapter is the whole picture: one pass through the system as a user experiences it, with the machinery behind each step referenced by chapter.

10.1 First run: from zero to first token

- **1. Owner starts the Agent** on the PC: TLS identity is created, migrations run, the Registry fills from configured backends (LM Studio at 127.0.0.1:1234, Ollama at 127.0.0.1:11434), and mDNS begins advertising (Ch. 5).
- **2. Owner opens a pairing session** in the admin console (:8444); the Agent shows a QR with a one-time secret (Ch. 7.2).
- **3. The app scans and confirms;** the Keystore key is born, the pin is verified, pairing completes zero-body, the owner approves, and the device receives its token (Ch. 7.2-7.4).
- **4. The app connects:** SM-CONN plans and races endpoints, wins on LAN, schedules the 80% token refresh, and lands CONNECTED (Ch. 7.5).
- **5. The user sends a prompt:** the composer builds the request, PinnedHttp streams the POST, the scheduler admits it, the adapter calls LM Studio on loopback, and SSE deltas flow back through SM-STREAM into the UI, closing with mesh.stats usage (Ch. 7.6).
- **6. Everything after is lifecycle:** backgrounding cancels streams; Wi-Fi to Tailnet handoff re-plans; a revoked device sees its stream torn down with a REVOKED reason; the Doctor explains any failure through the reason ladder (Ch. 7.7).

10.2 The developer loop

Development uses the same machinery with T0 conveniences: the run-book starts the two fake backends and the Agent with `--dev-insecure-loopback` (explicitly scoped, §17.9), and the dashboard renders the app's web export in a virtual-device panel against the real Agent - the same domain code, the same contracts, faster iteration. The full verified recipe lives in docs/handover/RUN-BOOK.md and Chapter 12 of this document.

10.3 What the cloud ever sees

If the owner enables the Control Plane, the Agent sends heartbeats, registry metadata, and revocation mirrors - identifiers and timings, never prompts or completions. With the Control Plane disabled, the system is fully LAN/Tailnet-local with no outbound cloud dependency at all. This is the architectural answer to the product's central promise.

11. Testing, QA, and Release Engineering

11.1 Test architecture

Layer (§21.5)	Location	What it proves
Unit	agent/tests/unit (32 modules)	Services, parsers, state logic, policy, packaging - fast and hermetic.
Contract	agent/tests/contract	Adapters against recorded real-backend fixtures - no mock drift.
Integration	agent/tests/integration + tools/fake-backends/tests	Full agent + fake backends: pairing, chat SSE, tasks, admin conformance, routing.
Security (§17.13)	agent/tests/security	Canary, SPAKE2+ vectors, replay TC-SEC-03, fuzz TC-SEC-05, artifact TC-SEC-07, CP schema TC-SEC-08, rate-limit TC-SEC-09, TLS 1.3 TC-SEC-10.
Mobile TS	apps/mobile/src/domain + packages/mesh-protocol	158 tests: state machines, planner, tokens, reasons - deterministic via simulated clock (§11.3, purity floor 80%).
Instrumented	apps/mobile/modules/mesh-core/androidTest	On-device: TLS pinning (incl. mismatch), NSD resolve.

Table 14: Test layers and what each one proves.

11.2 Quality gates on every change

The PR gate (staged at docs/ci/PR-GATE.yml, restored with one command per the README) runs, in §21.4 order: ruff lint and format, mypy --strict on core and security, the four pytest layers, the import-linter dependency rule (§10.1), the OpenAPI drift check, the four security scanners (cleartext, content columns, secrets, pip-audit), npm audit for the mobile project, and a JDK 17 gradlew assembleDebug build of the APK. Local developers run the same chain - every command in CONTRIBUTING.md was executed and verified before being documented.

11.3 Release engineering

Tagging triggers the release job: the full gate runs with --release (which additionally requires the on-device pentest sign-off to be present), release_build.sh produces the wheel, sdist, and SHA256SUMS, and the TC-SEC-07 scan inspects the artifacts. Signing is deliberately manual - the release key lives outside the repository and minisign signs the checksums (docs/RELEASE.md). v1.0.0-rc.1 shipped through this path with 14/14 checks green; the 1.0.0 final tag additionally requires the pentest sign-off and the §18.8 network matrix on real hardware.

12. Deployment and Operations

12.1 Running the Agent

```
cd agent
python3.12 -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt # pinned + hashed (17.11)
python -m localmesh_agent run --config config.toml # TLS 1.3 by default
# dev only (17.9, off by default): run --dev-insecure-loopback
```

Configuration (Appendix E of the spec) sets listen/admin ports, mDNS behavior, backend endpoints, and retention. The Agent prints its SPKI pin at startup; the QR shown during pairing embeds the same identity so the app can pin from day one.

12.2 Building the Android app

```
cd apps/mobile && bun install
npx expo prebuild --platform android --no-install # regenerate android/ if needed
cd android && ./gradlew assembleDebug # JDK 17 + Android SDK
# artifact: app/build/outputs/apk/debug/app-debug.apk (ai.localmesh.app)
```

The committed android/ directory builds as-is; prebuild regenerates it after native config changes. The debug keystore is the standard Android debug credential and is documented in SECURITY.md; release signing keys live outside the repository.

12.3 Operating the mesh

- **Pair a device:** admin console (:8444) - open pairing, scan, approve.
- **Manage devices:** list, rename, revoke from the admin API; revocation tears streams (§15.6).
- **Rotate TLS:** admin/tls/rotate with a backup-pin grace window; devices re-pin without re-pairing.
- **Diagnose:** the app's Doctor screen or admin/doctor walks §18.4 checks; the reason ladder names the failure.
- **Enable the Control Plane:** activate via ADR-019 flow (admin register); metadata-only guaranteed by schema scan.

12.4 Known open items before 1.0 final

Chapter 13 lists them; operationally the two that touch deployment are the on-device pentest sign-off (required for --release tagging of 1.0.0 final) and the §18.8 network matrix execution on the owner's hardware.

13. Open Items and Roadmap

The project keeps its open questions in one ledger (docs/OPEN_QUESTIONS.md) and never guesses owner decisions. The current state, after the Q-08 license resolution (Apache-2.0), is:

Item	Status	Next step
Pentest sign-off (§18.8 + checklist)	Open (owner)	Execute the checklist on real hardware; sign docs/security/pentest-signoff.md; unlocks 1.0.0 final tagging.
Real-network matrix (§18.8)	Open (owner)	Run LAN / Tailnet / firewall / VPN-conflict cases on the owner's devices.
ADR-017/018 approvals	Proposed	Owner ratifies the setuptools and nvidia-ml-py decisions already implemented.
CI workflow restoration	Staged	Push docs/ci/PR-GATE.yml to .github/workflows/ci.yml with a token holding the Workflows permission.
Q-09 distribution channel	Open (owner)	Sideload/F-Droid is the spec default for v1.0; Play Store stays optional.
Post-1.0 candidates	Backlog	Tier T3 relay (needs ADR), iOS port, Control Plane UI, deeper RAG tooling.

Table 15: Open items and roadmap at v1.0.0-rc.1.

Appendix A. API Endpoint Reference

A.1 Mesh API (/mesh/v1, TLS 8443)

Method	Path	Purpose
GET	/mesh/v1/info	Agent identity, version, pairing_open, capabilities.
GET	/mesh/v1/health	Liveness plus backend reachability summary.
GET	/mesh/v1/device	This device's registration and token state.
POST	/mesh/v1/pair/complete	Complete pairing (FR-PAIR-03 zero-body trust binding).
POST	/mesh/v1/pair/status	Poll pairing session state.
POST	/mesh/v1/auth/challenge	Device-token challenge exchange.
POST	/mesh/v1/auth/token	Issue/refresh device token.
GET	/mesh/v1/models	Capability Registry catalog (mesh_model_id space).
POST	/mesh/v1/models/load	Load a model on its backend.
POST	/mesh/v1/models/unload	Unload a model.
POST	/mesh/v1/chat/completions	Chat generation; SSE stream response.
DELETE	/mesh/v1/requests/{request_id}	Cancel an in-flight generation.
POST	/mesh/v1/tasks	Create a durable task (§13.9).

Method	Path	Purpose
GET	/mesh/v1/tasks/{task_id}	Task status/result.
DELETE	/mesh/v1/tasks/{task_id}	Delete a task before retention sweeps it.
GET	/mesh/v1/tasks/{task_id}/events	Per-task SSE event stream.
PUT	/mesh/v1/tasks/{task_id}/attachments/{name}	Upload an attachment (AES-GCM sealed at rest).

Table 16: Mesh API endpoints (docs/openapi/mesh-v1.json).

A.2 Admin API (loopback 8444)

Group	Endpoints
Pairing	POST /admin/pairing/open - GET /admin/pairing - GET /admin/pair/pending - POST /admin/pairing/approve - POST /admin/pairing/deny - POST /admin/pairing/close
Devices	GET /admin/devices - PATCH /admin/devices/{device_id} - DELETE /admin/devices/{device_id} - POST /admin/devices/{device_id}/revoke
TLS	POST /admin/tls/rotate - POST /admin/tls/backup-pin
Control Plane	POST /admin/control-plane/register - GET /admin/control-plane/status
Diagnostics	GET /admin/status - GET /admin/doctor - GET /admin/metrics

Table 17: Admin API endpoint groups (admin_app.py; 127.0.0.1 only).

Appendix B. Glossary

Term	Meaning (canonical vocabulary, AGENTS.md)
Mesh	The set of devices and Agents that trust each other through pairing.
Agent	The PC-side service exposing the Mesh API. Never called a server.
Backend	A local inference engine (LM Studio, Ollama, OpenAI-compatible, Whisper) reached loopback-only.
Mesh API	The /mesh/v1 HTTPS surface defined by docs/openapi/mesh-v1.json.
Content vs Metadata	Content = user prompts/completions/media (never leaves the mesh). Metadata = operational data that may be logged or mirrored.
Pairing Secret	One-time QR secret that bootstraps device trust; the only secret ever displayed.
Device Token	Opaque per-device bearer credential; stored hashed; refresh at 80% lifetime.
SPKI pin	b64url(SHA-256) of the Agent TLS leaf key; stored per agent; verified on every connection.
Transport Tier (T0-T3)	Loopback dev / LAN / Tailscale / future relay - stored per endpoint and raced.
mesh_model_id	Stable model reference of the form backend_id::backend_model_id.
Request vs Task	A streaming generation vs a durable workload with attachments and retention.
SAS	Software Architecture Specification - LM-ARCH-001, the governing document.

Term	Meaning (canonical vocabulary, AGENTS.md)
Connection Manager (SM-CONN)	App reducer owning connectivity state and endpoint selection.
Doctor	Ordered connectivity diagnostic (§18.4) shared by app screen and admin API.
Reason ladder	§18.3 aggregation of probe failures into one actionable ReasonCode.

This reference was generated from the LocalMesh-App repository at v1.0.0-rc.1 (main). The governing specification LM-ARCH-001 and the ADR log remain the authoritative sources; where this document and the spec disagree, the spec wins.